

07-docker-realtime.md

REAL-WORLD MAP (GitHub vs Docker Hub)

Think like this:

Concept	Git World 	Docker World 
Code storage	GitHub	Docker Hub
What we store	Source Code	Docker Images
Clone	<code>git clone</code>	<code>docker pull</code>
Push	<code>git push</code>	<code>docker push</code>

Memory Trick:

 *GitHub = code*

 *Docker Hub = app ready-to-run (image)*

FULL FLOW (REAL WORLD STORY)

Imagine:

-  Developer gives code →
-  DevOps (you) prepares package →
-  Store it in warehouse →
-  Anyone can use it

Mapping:

1. Developer gives code
 2. You write Dockerfile
 3. Build image
 4. Push to Docker Hub
 5. Anyone can pull & run
-

STEP-BY-STEP FLOW (WITH COMMAND MAP)

1. Take Code

```
git clone https://github.com/Harishjangidi/staticsite-docker.git  
cd staticsite-docker/html
```

👉 Like downloading project from GitHub

2. Create Dockerfile

```
FROM nginx  
COPY . /usr/share/nginx/html
```

👉 Like packing files into a box 📦

3. Build Image

```
docker build -t image1 .
```

👉 Converts code → Docker Image

4. Tag Image (VERY IMPORTANT 🔥)

```
docker tag image1 jangidiharish07924/test-stats-docker:latest
```

👉 Mapping:

local-image → dockerhub-username/repo

5. Login to Docker Hub

```
docker login
```

6. Push Image

```
docker push jangidiharish07924/test-stats-docker:latest
```

👉 Now image is stored in cloud ☁️

📦 TYPES OF DOCKER REGISTRY

1. Cloud Registry

- Docker Hub (default)
- GCR

- AWS ECR

👉 Like Google Drive ☁️

2. Local Registry

- Nexus
- JFrog
- DTR

👉 Like your local hard disk 💻

⚙️ JENKINS PIPELINE (MAP + SIMPLE)

🧩 Stage 1: Code

git "repo-url"

👉 Get code

🧩 Stage 2: Build

docker build -t image .

👉 Create image

🧩 Stage 3: Deploy

docker run -d -p 8082:80 image

👉 Run container

🧩 Stage 4: Push (IMPORTANT TASK 🔥)

Add this:

```
stage("Push") {  
  steps {  
    sh '''  
    docker login  
    docker tag pavan:one jangidiharish07924/test-stats-docker:latest  
    docker push jangidiharish07924/test-stats-docker:latest  
    '''  
  }  
}
```

```
}  
}
```



FINAL PIPELINE FLOW (EASY MEMORY)

👉 **Code** → **Build** → **Deploy** → **Push**

GitHub → Docker Image → Container → Docker Hub



JENKINS + DOCKER INTEGRATION (WHY?)

👉 To see:

- Running containers
- Images

Steps:

1. Install Docker plugin
2. Enable Docker API
3. Add Docker cloud in Jenkins

👉 Result: Jenkins can talk to Docker 🧠



ONE-LINE MEMORY

👉 **DevOps job** =

Take code → Build image → Run container → Push to Docker Hub



ELI5 (Like 5th Class Kid)

Imagine:

- GitHub = Book 📖
- Docker = Lunch box 🍱
- Docker Hub = Store 🏪

Steps:

1. Take recipe (code)

2. Cook food (build image)
 3. Pack in lunchbox (image)
 4. Put in store (Docker Hub)
 5. Anyone can take and eat (run container)
-

ELI10 (Slight Technical)

- Developer writes code
- DevOps creates Dockerfile
- Dockerfile builds image
- Image is pushed to Docker Hub
- Jenkins automates everything

👉 Pipeline:

Git → Build → Run → Push

FINAL PRESENTATION LINE (USE THIS)



“Docker Hub is like GitHub but for images.

We take code, build Docker image, and push it to Docker Hub so anyone can pull and run the application anywhere.”

👉 Just remember:

INTERVIEW QUESTIONS (WITH SIMPLE ANSWERS)

What is Docker Hub?



“Docker Hub is a cloud registry where we store and share Docker images, similar to GitHub but for images.”

Why tagging is required?



“Tagging helps identify image version and connects local image to Docker Hub repository.”

? Difference: Image vs Container?



- Image = Template 📦
 - Container = Running app 🚀
-

? Why Jenkins?



“To automate build, deploy, and push process without manual work.”